

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №2
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Оценка времени выполнения программ»

Выполнили студенты группы 25BBB2

Лампетов А.С.

Ковтун А.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

Тема: Оценка времени выполнения программ

Цель работы: научиться оценивать время выполнения программ.

Лабораторные задания:

1. Вычислить порядок сложности программы, вычисляющей произведение двух матриц.
2. Оценить время выполнение программы, вычисляющей произведение двух матриц, для матриц размером 100, 200, 400, 1000, 2000, 4000, 10000.
3. Оценить время работы алгоритмов сортировки Шелла и быстрой сортировки.

1. Порядок сложности программы, вычисляющей произведение двух матриц, равен количеству операций внешнего цикла, умноженных на количество операций среднего цикла, умноженных на количество операций внутреннего цикла. Все циклы имеют сложность $O(n)$. Следовательно сложность программы равна $O(n) * O(n) * O(n) = O(n^3)$.

Аналитический график зависимости времени выполнения от N

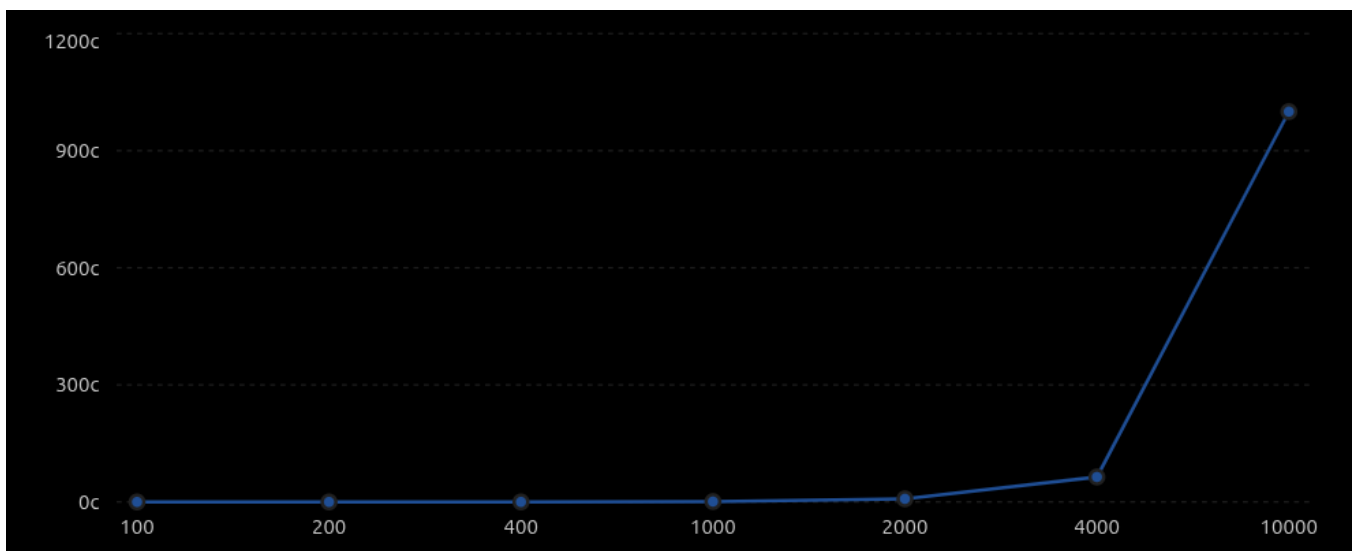


Рисунок 1 — Аналитический график зависимости времени выполнения от N

Время выполнения программ для матриц размерами 100, 200, 400, 1000, 2000, 4000, 10000. Результаты работы программы:

```
andrey@andrey-CNP6S:~/Project_C++/algorithms
→ algorithms git:(laba2) X go build -o mainGo.exe main.go
→ algorithms git:(laba2) X ./mainGo.exe
Size: 100 x 100
Time taken for matrix multiplication: 0.00084 seconds
→ algorithms git:(laba2) X go build -o mainGo.exe main.go
→ algorithms git:(laba2) X ./mainGo.exe
Size: 200 x 200
Time taken for matrix multiplication: 0.00414 seconds
→ algorithms git:(laba2) X go build -o mainGo.exe main.go
→ algorithms git:(laba2) X ./mainGo.exe
Size: 400 x 400
Time taken for matrix multiplication: 0.02415 seconds
→ algorithms git:(laba2) X go build -o mainGo.exe main.go
→ algorithms git:(laba2) X ./mainGo.exe
Size: 1000 x 1000
Time taken for matrix multiplication: 0.12752 seconds
→ algorithms git:(laba2) X go build -o mainGo.exe main.go
→ algorithms git:(laba2) X ./mainGo.exe
Size: 2000 x 2000
Time taken for matrix multiplication: 19.95706 seconds
→ algorithms git:(laba2) X go build -o mainGo.exe main.go
→ algorithms git:(laba2) X ./mainGo.exe
Size: 4000 x 4000
Time taken for matrix multiplication: 298.47303 seconds
→ algorithms git:(laba2) X go build -o mainGo.exe main.go
→ algorithms git:(laba2) X ./mainGo.exe
Size: 10000 x 10000
Time taken for matrix multiplication: 6118.65336 seconds
→ algorithms git:(laba2) X

andrey@andrey-CNP6S:~/Project_C++/algorithms
→ algorithms git:(laba2) X ./mainCPP.exe
Size: 100 x 100
Time taken for matrix multiplication: 0.005578 seconds
→ algorithms git:(laba2) X g++ -o mainCPP.exe main.cpp
→ algorithms git:(laba2) X ./mainCPP.exe
Size: 200 x 200
Time taken for matrix multiplication: 0.020583 seconds
→ algorithms git:(laba2) X g++ -o mainCPP.exe main.cpp
→ algorithms git:(laba2) X ./mainCPP.exe
Size: 400 x 400
Time taken for matrix multiplication: 0.142546 seconds
→ algorithms git:(laba2) X g++ -o mainCPP.exe main.cpp
→ algorithms git:(laba2) X ./mainCPP.exe
Size: 1000 x 1000
Time taken for matrix multiplication: 2.58452 seconds
→ algorithms git:(laba2) X g++ -o mainCPP.exe main.cpp
→ algorithms git:(laba2) X ./mainCPP.exe
Size: 2000 x 2000
Time taken for matrix multiplication: 25.0427 seconds
→ algorithms git:(laba2) X g++ -o mainCPP.exe main.cpp
→ algorithms git:(laba2) X ./mainCPP.exe
Size: 4000 x 4000
Time taken for matrix multiplication: 246.366 seconds
→ algorithms git:(laba2) X g++ -o mainCPP.exe main.cpp
→ algorithms git:(laba2) X ./mainCPP.exe
Size: 10000 x 10000
Time taken for matrix multiplication: 5871.69 seconds
→ algorithms git:(laba2) X
```

Рисунок 2 — Результаты работы программы

График зависимости выполнения программы от размера матрицы:



Рисунок 3 — График зависимости выполнения программы от размера матрицы

Таблица результатов			
Размер матрицы N	Go, с	C++, с	Разница, раз
100	0.00084	0.005578	6.64
200	0.00414	0.020583	4.97
400	0.02415	0.142546	5.90
1000	0.12752	2.58452	20.27
2000	19.95706	25.04270	1.25
4000	298.47303	246.36600	1.21
10000	6118.65336	5871.69000	1.04

Рисунок 4 — Таблица результатов

В ходе эксперимента было исследовано время выполнения алгоритма умножения квадратных матриц на языках Go и C++. При увеличении размера матрицы время выполнения обеих программ значительно возрастает, что соответствует теоретической вычислительной сложности алгоритма $O(N^3)$. На небольших размерах матриц реализация на Go показала меньшее время выполнения, однако с увеличением размера матрицы различие между реализациями постепенно уменьшается. При $N=4000$ и $N=10000$ реализация на C++ показывает немного меньшее время выполнения. При $N=10000$ время работы составило 6118,65 с для Go и 5871,69 с для C++. Таким образом, экспериментальные результаты подтверждают кубический характер роста времени выполнения, хотя конкретные значения отличаются от идеальной теоретической модели из-за особенностей работы процессора, памяти и компилятора.

2. Оценили время алгоритмов сортировки Шелла, быстрой сортировки и стандартной функции qsort на массиве размером 1000.

Сравнение алгоритмов сортировки (N = 1000)			
Тип последовательности	Shell	QS	qsort
Случайные значения	0.000300	0.000151	0.000189
Возрастающая последовательность	0.000030	0.000046	0.000058
Убывающая последовательность	0.000418	0.000043	0.000062
Возрастающая-убывающая последовательность	0.000246	0.000843	0.000064

Оценили время алгоритмов сортировки Шелла, быстрой сортировки и стандартной функции qsort на массиве размером 100000.

Сравнение алгоритмов сортировки (N = 100000)			
Тип последовательности	Shell	QS	qsort
Случайные значения	0.398283	0.004000	0.005641
Возрастающая последовательность	0.000685	0.001413	0.001952
Убывающая последовательность	0.815816	0.001224	0.002300
Возрастающая-убывающая последовательность	0.410764	2.074128	0.002251

В ходе эксперимента было оценено время работы алгоритмов сортировки Шелла, быстрой сортировки и стандартной функции qsort на массивах размером 1000 и 100000 элементов. Результаты показали, что время выполнения зависит не только от размера массива, но и от характера исходной последовательности. Для массива из 1000 элементов на случайной и убывающей последовательностях наименьшее время показала быстрая сортировка, на возрастающей последовательности - сортировка Шелла, а на возрастающей-убывающей последовательности - стандартная функция qsort. При увеличении размера массива до 100000 элементов различия между алгоритмами стали значительно более выраженными. Быстрая сортировка показала наименьшее время на случайной и убывающей последовательностях, сортировка Шелла - на возрастающей последовательности, а qsort - на возрастающей-убывающей последовательности. Таким образом, по полученным экспериментальным данным нельзя выделить один алгоритм, который был бы самым быстрым для всех типов входных данных: время работы существенно зависит от структуры исходного массива.

Вывод по работе: В ходе работы были освоены навыки оценивания времени выполнения программ.